

# Microservices Deployment with Containers and Orchestrators

## Introduzione

### Il problema ereditato

*"Distribuire microservizi in qualsiasi momento con zero downtime e scala flessibile. I metodi tradizionali di distribuzione manuale **non** possono soddisfare queste esigenze."*

### I tre requisiti:

| Tipo        | Esigenza                                                          |
|-------------|-------------------------------------------------------------------|
| Business    | "Spedire funzionalità più velocemente e innovare continuamente"   |
| Operativi   | "Distribuire aggiornamenti <b>senza interrompere gli utenti</b> " |
| Scalabilità | "Gestire in modo affidabile <b>l'enorme traffico fluttuante</b> " |

### La roadmap

1. **Container** (Docker) — "packaging standardizzato con tutte le dipendenze"
2. **Orchestratori** (Kubernetes) — "automatizzano deployment, scaling, self-healing e networking"
3. **Strategie di deployment** — blue-green, rolling, canary
4. **Pattern Kubernetes** — sidecar e service mesh
5. **DevOps, CI/CD, Infrastructure as Code, GitOps**

---

## What are Containers?

### Definizione

Una scatola standardizzata e autonoma per la propria applicazione: impacchetta il codice del microservizio insieme a **TUTTE** le sue dipendenze — runtime, librerie di sistema, file di configurazione, strumenti.

★ *"Risolve il classico problema degli sviluppatori — **"ha funzionato sulla mia macchina"** — perché il container include tutte le dipendenze. La stessa immagine può essere eseguita in modo **IDENTICO** sul laptop di uno sviluppatore, su un server di test, o nel cloud di produzione."*

### L'analogia:

*"Un container marittimo ha **dimensione e formato standard**, quindi può essere spostato da qualsiasi nave, treno o gru, **indipendentemente da ciò che contiene**. Allo stesso modo, un container software fornisce **un formato standard per le applicazioni**."*

---

## ★ Container vs Macchine Virtuali

|                   | Macchina Virtuale                                                                                            | Container                                                                           |
|-------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Cosa virtualizza  | L'hardware fisico (tramite hypervisor)                                                                       | Il sistema operativo                                                                |
| Sistema operativo | ⚠ "Ogni VM include la propria copia <b>COMPLETA</b> di un sistema operativo: kernel, file di sistema, tutto" | ✅ "Tutti i container condividono <b>il kernel del sistema operativo dell'host</b> " |
| Dimensione        | Molto grandi (GB)                                                                                            | Leggeri (MB)                                                                        |
| Avvio             | "Lente ad avviarsi, spesso minuti"                                                                           | "Secondi o frazioni di secondo"                                                     |
| Densità           | Poche per host                                                                                               | "Molti più container su un singolo host"                                            |

Le due tecnologie del kernel Linux che rendono possibile l'isolamento:

- Namespaces — "forniscono l'isolamento dei processi, della rete e del file system"
  - cgroups — "limitano la quantità di CPU e memoria che ciascun container può utilizzare"
- "Questa natura leggera e veloce è ciò che li rende così potenti per i microservizi."

---

### I 3 termini chiave dell'ecosistema

| Termine            | Definizione                                                                                                                                                                                                                |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Container Image    | "Il progetto (blueprint) del container. Un pacchetto <b>leggero, immutabile, standalone ed eseguibile</b> che include <b>tutto il necessario</b> : codice, runtime, librerie. Si costruisce una volta e si esegue ovunque" |
| Container Runtime  | "Il motore sottostante responsabile dell'esecuzione effettiva. <b>Estrae l'immagine, crea il processo del container e ne gestisce isolamento e risorse</b> . Esempi: containerd, CRI-O"                                    |
| Container Registry | "Un sistema di archiviazione per le immagini. Come una biblioteca. <b>Crei l'immagine e la invii (push) al registry; il server di produzione la estrae (pull)</b> "                                                        |

Registry: Docker Hub (pubblico), Amazon ECR, Azure ACR, Google GCR (privati).

---

## ★ I 4 motivi per cui i container sono perfetti per i microservizi

### 1. Isolamento del servizio

"Collocando ciascun microservizio nel proprio container, **otteniamo un forte isolamento dei processi e delle dipendenze a livello di sistema operativo.**"

### 2. Gestione delle dipendenze — "un enorme vantaggio"

"Il container di Catalog può contenere **la versione specifica di .NET e le librerie di cui ha bisogno**. Un microservizio separato — Discount o Ordering — potrebbe contenere una versione **DIVERSA** di .NET, o essere scritto in un linguaggio diverso come Java. **Non entreranno in conflitto anche quando eseguiti sulla stessa macchina host.**"

Questo è ciò che rende realmente possibile la **polyglot architecture** della Lezione 5.

### 3. Portabilità e coerenza

"La **STESSA immagine** può essere usata per i test locali, l'integrazione e il deployment in produzione."

## 4. Scalabilità efficiente

*"Quando un microservizio registra un'impennata di traffico, un orchestratore può avviare nuove istanze in pochi SECONDI, e chiuderle altrettanto rapidamente quando il picco è terminato."*

---

## What is Docker?

*"Mentre i container sono un concetto, Docker è una piattaforma tecnologica e un insieme di strumenti che rendono la containerizzazione pratica e semplice."*

### L'analogia estesa:

*"Se un container è la scatola di spedizione standardizzata, Docker fornisce le gru, i camion e le navi — l'intero sistema logistico per costruire la scatola, spostarla e disimballarla ovunque debba andare."*

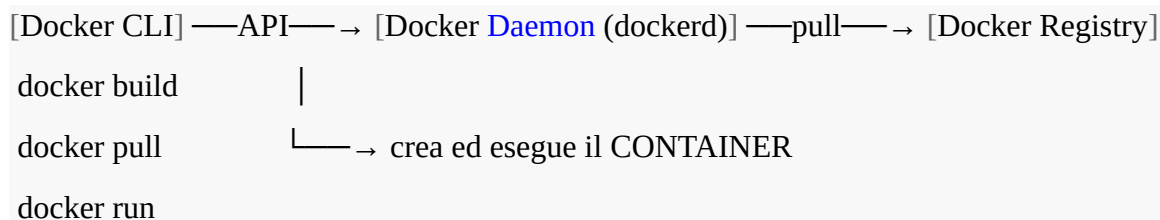
*"L'obiettivo principale è separare le applicazioni dall'infrastruttura."*

### I 3 componenti principali

| Componente       | Definizione                                                                                                                                                                     | Analogia                                                           |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Docker Image     | "Modello immutabile di sola lettura che contiene tutto per eseguire l'applicazione. Si crea scrivendo un file di testo chiamato <b>Dockerfile</b> , con istruzioni passo-passo" | "Una <b>ricetta</b> , o una <b>classe</b> nella programmazione OO" |
| Docker Container | "Un'istanza <b>IN ESECUZIONE</b> di un'immagine. Se l'immagine è il progetto, il container è la casa costruita a partire da esso. È il processo dal vivo"                       | "Un <b>oggetto</b> istanziato dalla classe"                        |
| Docker Registry  | "Sistema di archiviazione e distribuzione delle immagini. <b>Come GitHub per le immagini</b> "                                                                                  | —                                                                  |

*"È possibile eseguire più container dalla stessa immagine, ciascuno isolato dagli altri."*

### L'architettura client-server



| Componente             | Ruolo                                                                                                                                                  |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Docker Client          | "Lo strumento a riga di comando dove si digitano <code>docker build</code> , <code>docker pull</code> , <code>docker run</code> "                      |
| Docker Daemon (Engine) | "Servizio persistente in background sulla macchina host. Fa il lavoro pesante: ascolta le richieste API e gestisce immagini, container, reti e volumi" |
| Docker Registry        | Dove sono memorizzate le immagini                                                                                                                      |

## Il flusso di docker run:

1. Il client invia il comando al daemon tramite API
2. Il daemon **controlla se ha l'immagine in locale**
3. **In caso contrario, la preleva (pull) dal registry**
4. **Crea ed esegue un nuovo container** da quell'immagine

## Il processo di containerizzazione

1. Scrivi il DOCKERFILE
  2. docker build → crea l'IMMAGINE
  3. docker push → invia l'immagine al REGISTRY
  4. docker run → esegui il CONTAINER su qualsiasi macchina
- 

## What are Container Orchestrators?

### Definizione

**Uno strumento che automatizza l'intero ciclo di vita delle applicazioni containerizzate su un intero cluster di macchine.**

*"Invece di accedere manualmente ai server per avviare, arrestare o aggiornare i container, l'orchestratore li gestisce per voi."*

### L'analogia:

*"Un esperto gestore di flotte. Non dici a una nave specifica dove andare: dici al gestore "ho bisogno di consegnare questo carico", e lui si occupa di assegnare le navi, tracciarle e risolvere i problemi."*

### ★ La configurazione dichiarativa

*"Con un orchestratore non si posizionano manualmente i container su server specifici. Si fornisce una CONFIGURAZIONE DICHIARATIVA che descrive lo STATO DESIDERATO."*

*"Ad esempio: "voglio TRE istanze del microservizio Basket in esecuzione e disponibili su questo indirizzo di rete"."*

★ *"Il compito dell'orchestratore è LAVORARE COSTANTEMENTE affinché lo stato EFFETTIVO del cluster corrisponda allo stato DESIDERATO."*

Questa è la differenza fondamentale tra **imperativo** ("fai questo, poi quello") e **dichiarativo** ("questo è il risultato che voglio, arrangiati").

### Le 6 domande che risolve

*"Pensate di avere centinaia di microservizi:"*

1. *"Come vengono fornite le istanze dei container su cluster di più macchine?"*
2. *"Come faranno i container a scoprirsi e comunicare tra loro?"*

3. *"Come possono scalare in base alla domanda e ai picchi di traffico?"*
4. *"Come monitorare la salute di ogni container?"*
5. *"Come proteggerli dai guasti hardware e software?"*
6. *"Come aggiornarli per un'applicazione live con ZERO downtime?"*

*"Queste attività non possono essere gestite manualmente per migliaia di container."*

**Orchestratori:** Kubernetes ("di gran lunga il più diffuso e dominante"), Docker Swarm, Apache Mesos.

---

## What is Kubernetes? Architecture, Components, Helm Charts

### Definizione

Kubernetes (abbreviato K8s — "perché tra la K e la S ci sono otto lettere") è una piattaforma open source di orchestrazione dei container.

*"Originariamente sviluppato da **Google**, ora supportato dalla **Cloud Native Computing Foundation**. Consente di **gestire gruppi di server** — che chiama **CLUSTER** — come una singola unità logica."*

*"Si basa sulla **configurazione dichiarativa**: si fornisce un file di configurazione, in genere in formato **YAML**, che descrive lo stato desiderato."*

*"È **agnostico rispetto alla piattaforma**: cluster on-premise o su AWS, Azure, Google Cloud."*

---

### ★ Le 4 funzionalità che risolvono le sfide operative

#### 1. Self-Healing (autoguarigione)

*"Kubernetes **monitora costantemente i container**. Se un container si blocca, lo **RIAVVIA automaticamente**. Se l'**INTERO NODO** si guasta, **RIPROGRAMMA automaticamente il container su un altro nodo del cluster**."*

#### 2. Auto-scaling

*"Può **scalare automaticamente i microservizi in orizzontale**. Si può configurare per controllare l'utilizzo di CPU o memoria: se supera l'80%, Kubernetes **AGGIUNGE automaticamente altre istanze**. Quando il carico diminuisce, le rimuove."*

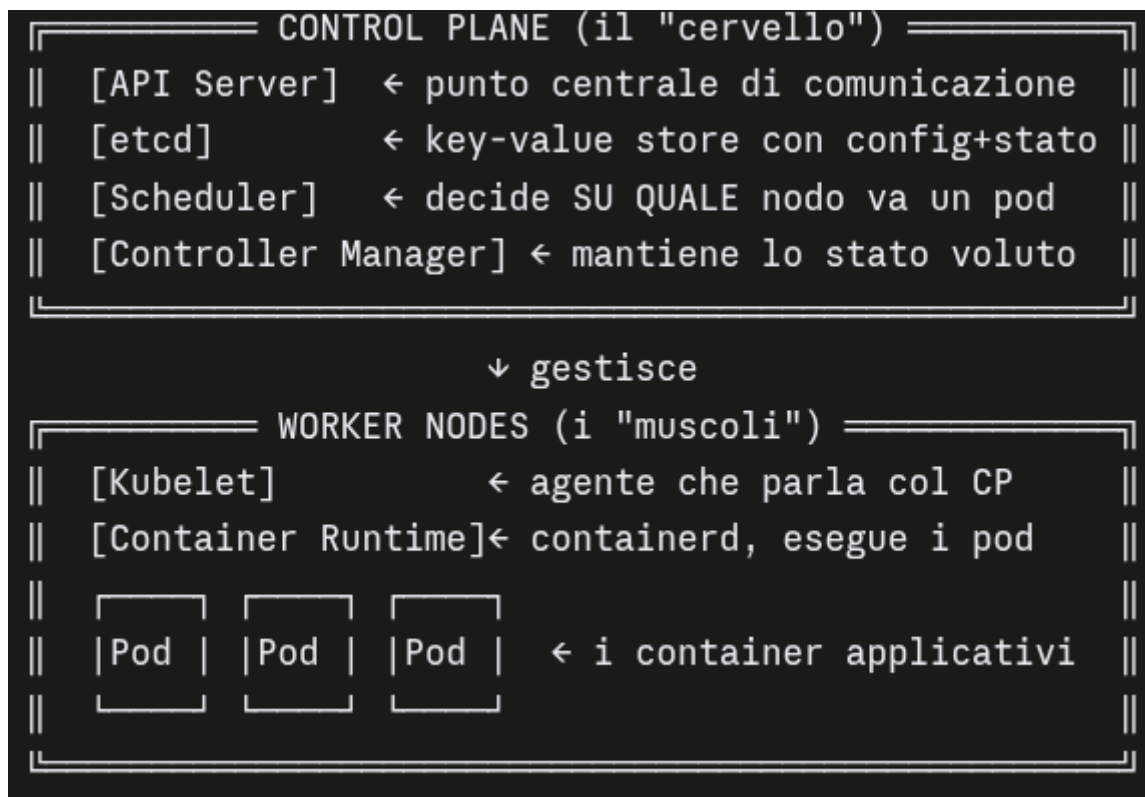
#### 3. Service Discovery e Load Balancing ★

*"Kubernetes fornisce a un insieme di container **un singolo NOME DNS STABILE e un INDIRIZZO IP STABILE**, e **bilancia automaticamente il traffico tra tutte le istanze SANE**."*

Questo risolve nativamente il **Service Registry/Discovery pattern** della Lezione 8 — motivo per cui la lezione diceva che oggi raramente serve Consul o Eureka.

#### 4. Rollout e Rollback automatizzati

*"Si dice a Kubernetes di aggiornare il servizio a una nuova immagine, **usando strategie come i rolling update per garantire zero downtime**. E se qualcosa va storto, si comanda il ritorno alla versione stabile precedente."*



| Componente         | Ruolo                                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------|
| API Server         | "Il punto centrale di tutte le comunicazioni"                                                                                    |
| etcd               | "Archivio key-value <b>coerente e altamente disponibile</b> dove sono memorizzati <b>configurazione e stato del cluster</b> "    |
| Scheduler          | "Decide su quale worker node deve essere eseguito un container"                                                                  |
| Controller Manager | "Esegue vari controller <b>per mantenere lo stato desiderato</b> "                                                               |
| Kubelet            | "Agente che <b>comunica con il control plane</b> e si assicura che i container siano in esecuzione su quel nodo come dovrebbero" |
| Container Runtime  | "Il software di basso livello che <b>esegue effettivamente i container</b> "                                                     |

## ★ I 6 oggetti principali di Kubernetes

### 1. Pod

*"L'unità più piccola e semplice da distribuire in Kubernetes. Rappresenta **una singola istanza dell'applicazione**. Può contenere **UNO O PIÙ container strettamente accoppiati che condividono risorse di rete e storage**."*

### 2. ReplicaSet

*"Garantisce che un numero specifico di pod identici (repliche) sia SEMPRE in esecuzione. Se un pod si guasta, ne crea uno nuovo per sostituirlo."*

### 3. Deployment ★

*"In genere non si creano direttamente pod o ReplicaSet. Si crea invece un Deployment: un oggetto di livello superiore che gestisce i ReplicaSet e fornisce aggiornamenti dichiarativi."*

*"Si dice al Deployment: "voglio 3 repliche del microservizio Catalog usando questa versione dell'immagine", e il Deployment si occupa di creare ReplicaSet e pod per raggiungere quello stato."*

Deployment → ReplicaSet → Pod, Pod, Pod

(cosa voglio) (quante copie) (istanze reali)

### Service ★

*"Fornisce un endpoint di rete STABILE per un insieme di pod. Poiché i pod possono essere creati e distrutti e i loro IP sono EFFIMERI, un Service fornisce un singolo nome DNS e un IP stabile, e bilancia il traffico tra tutti i pod sani."*

*"Questo è il modo in cui il service discovery è implementato in Kubernetes."*

### 5. ConfigMap e Secret

*"Permettono di disaccoppiare la configurazione dall'immagine del container. ConfigMap per i dati non sensibili; Secret per i dati sensibili come password o chiavi API."*

### 6. Volume

*"I container sono effimeri, ma spesso i dati devono persistere. I volumi forniscono un modo per collegare storage persistente ai pod."*

---

## ★ Helm Charts: il package manager per Kubernetes

### Il problema:

*"Un SINGOLO microservizio può richiedere diversi file di configurazione: un YAML di Deployment, un YAML di Service, una ConfigMap, un Secret. Ora moltiplicate per decine di microservizi. La gestione di tutti questi file YAML può essere un incubo."*

### La soluzione:

*"Helm è il package manager per Kubernetes. Consente di raggruppare tutti i file di configurazione relativi a un'applicazione in un unico pacchetto coesivo, chiamato Helm Chart."*

### I 3 vantaggi:

| Vantaggio       | Dettaglio                                                                                                |
|-----------------|----------------------------------------------------------------------------------------------------------|
| Un comando solo | "Automatizza la distribuzione dell'intera applicazione con un unico comando: <code>helm install</code> " |

| Vantaggio                        | Dettaglio                                                                                                                          |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Versionamento                    | "I chart sono <i>basati sulle versioni</i> , quindi è facile <i>eseguire upgrade o ROLLBACK a una versione precedente</i> "        |
| Personalizzazione per ambiente ★ | "Si usa <i>UN SOLO chart</i> e si <i>forniscono file di VALUES diversi</i> per configurarlo per <i>dev, staging o produzione</i> " |

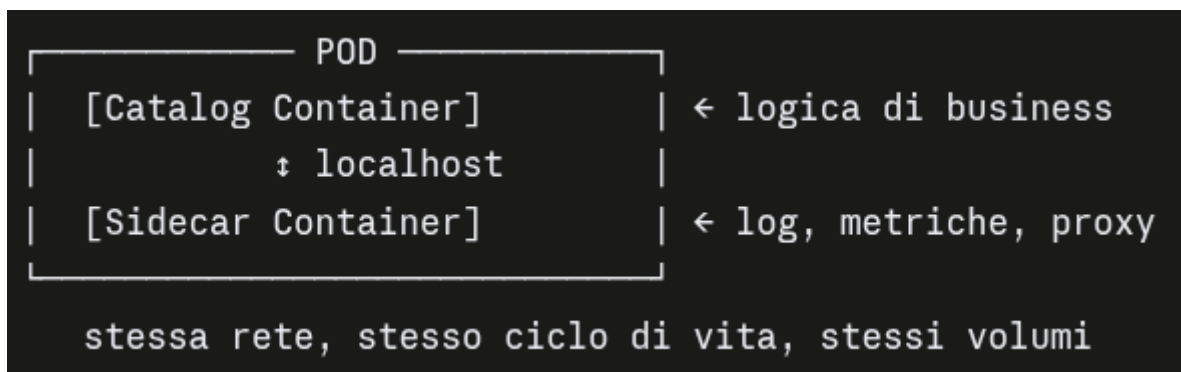
## Sidecar and Service Mesh Patterns

### Pattern Sidecar

Eseguire un container AUSILIARIO accanto al container principale dell'applicazione, all'interno dello STESSO pod Kubernetes.

L'analogia:

*"Come il sidecar di una moto: un'unità separata collegata alla moto principale, che condivide lo stesso viaggio ma ha uno scopo proprio."*



*"Poiché entrambi si trovano nello stesso pod, condividono lo stesso namespace di rete: possono comunicare tramite localhost. Condividono lo stesso ciclo di vita — vengono creati e distrutti insieme — e possono condividere i volumi."*

I 4 motivi per usarlo:

| Motivo                                                                                                                                                                               | Dettaglio                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Separation of concerns ★                                                                                                                                                             | "Il vantaggio più grande. Il team può concentrarsi <i>esclusivamente sulla logica di business</i> . Non devono incorporare codice complesso per logging, metriche o policy di rete." |
| Riutilizzabilità                                                                                                                                                                     | "Si crea <i>un sidecar standardizzato</i> — es. un agente di logging preconfigurato — e si <i>riutilizza lo stesso per TUTTI i microservizi</i> "                                    |
| Indipendenza linguistica                                                                                                                                                             | "Sidecar e applicazione principale possono essere <i>realizzati con tecnologie completamente diverse</i> "                                                                           |
| Aggiornamento indipendente                                                                                                                                                           | "Si <i>aggiorna l'immagine del sidecar</i> e si <i>ridistribuiscono i pod SENZA toccare o ricostruire il codice dell'applicazione</i> "                                              |
| ⚠ Il trade-off: "aggiunge complessità — <i>più container da gestire</i> — e <i>aumenta l'uso di risorse di ogni pod</i> , poiché il sidecar ha bisogno della propria CPU e memoria." |                                                                                                                                                                                      |

## ★ Service Mesh

Il pattern sidecar applicato IN SCALA all'intera applicazione, in particolare per gestire la comunicazione TRA servizi.

*"Crea un livello di infrastruttura dedicato, visibile e gestibile per controllare come i microservizi parlano tra loro."*

I 2 componenti:

| Componente    | Ruolo                                                                                                                                                                                                                                                                |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data Plane    | <i>"Una rete di proxy sidecar ad alte prestazioni — il più diffuso è Envoy. Un proxy viene distribuito accanto a OGNI singola istanza di microservizio. TUTTO il traffico in entrata e uscita viene intercettato e instradato attraverso il suo proxy dedicato."</i> |
| Control Plane | <i>"Il cervello centrale. Si definiscono policy — regole di routing, sicurezza, logica di retry — e il control plane CONFIGURA AUTOMATICAMENTE tutti i proxy sidecar per applicarle."</i>                                                                            |

★ *"In questo modo il codice dell'applicazione si limita a fare una semplice richiesta di rete, e il proxy sidecar gestisce in modo TRASPARENTE tutta la complessità."*

Le 4 capacità del service mesh

| Capacità              | Esempio                                                                                                                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gestione del traffico | <i>""Invia il 90% del traffico alla v1 di Catalog e il 10% alla v2" per un rilascio canary. Il sidecar lo applica automaticamente"</i>                                                               |
| Resilienza ★          | <i>""Per qualsiasi chiamata a Ordering, riprova fino a 3 volte se fallisce, o apri un circuit breaker se non risponde." Gli sviluppatori NON devono implementare questa logica in ogni servizio"</i> |
| Sicurezza             | <i>"Cripa automaticamente TUTTO il traffico tra microservizi con mutual TLS (mTLS). Si possono definire policy a grana fine: "solo Ordering può chiamare Payment"""</i>                              |
| Osservabilità ★       | <i>"Poiché tutto il traffico passa attraverso i proxy, il mesh genera automaticamente metriche dettagliate, log e tracce distribuite per OGNI singola richiesta"</i>                                 |

Implementazioni: Istio, Linkerd.

Nota quanto è potente: il service mesh fornisce **gratuitamente e senza modificare il codice** molte delle cose che la Lezione 15 insegnerà a implementare a mano (retry, circuit breaker, tracing).

---

## DevOps and CI/CD Pipelines

Cos'è DevOps

★ *"DevOps NON è uno strumento né un titolo di lavoro. È una filosofia CULTURALE."*

*"L'obiettivo è abbattere i muri tradizionali tra i team di sviluppo — che vogliono inviare rapidamente nuove funzionalità — e i team operativi IT — che vogliono*

garantire la stabilità, **enfaticizzando collaborazione, responsabilità condivisa e automazione.**"

★ "In un'architettura a microservizi, ci sono **piccoli team autonomi responsabili dell'INTERO ciclo di vita del loro servizio.** "You build it, you run it" è un principio fondamentale della mentalità DevOps."

## CI/CD

**"Se DevOps è una cultura, CI/CD è il MOTORE AUTOMATIZZATO che la rende possibile."**

### Definizione

|                                    |                                                                                                                                                                                                                                                                                      |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Continuous Integration (CI)</b> | "Gli sviluppatori <b>uniscono frequentemente le modifiche in un repository centrale.</b> Ogni modifica avvia <b>un processo automatizzato che costruisce l'applicazione ed esegue test automatici.</b> L'obiettivo è <b>trovare e risolvere i bug di integrazione PRECOCEMENTE</b> " |
| <b>Continuous Delivery</b>         | "Una volta superati i test, <b>prepara automaticamente una release e la distribuisce in un ambiente di STAGING.</b> La distribuzione finale in produzione potrebbe richiedere la pressione <b>MANUALE</b> di un pulsante"                                                            |
| <b>Continuous Deployment</b> ★     | "La forma più avanzata: <b>distribuisce automaticamente OGNI modifica che supera tutte le fasi direttamente in PRODUZIONE,</b> senza alcun intervento umano"                                                                                                                         |

**"Per i microservizi, CI/CD non è opzionale, è essenziale. Immaginate di costruire, testare e distribuire manualmente CENTINAIA di microservizi, ognuno con un proprio programma di rilascio. Sarebbe impossibile."**

## Le 6 fasi della pipeline

1. SOURCE → lo sviluppatore fa push su Git
2. BUILD → CI compila il codice E CREA UN'IMMAGINE DOCKER versionata
3. TEST → test unitari + test di integrazione
4. PUSH → l'immagine viene taggata e inviata al CONTAINER REGISTRY
5. DEPLOY → aggiorna il Deployment Kubernetes con il nuovo tag immagine  
→ innesca un ROLLING UPDATE controllato
6. MONITOR → gli strumenti osservano la nuova versione per errori o problemi

**"Un risultato cruciale della fase di build è la creazione di un'IMMAGINE DOCKER versionata che impacchetta il servizio e tutte le sue dipendenze."**

## La toolchain tipica

| Fase                   | Strumenti                                                                    |
|------------------------|------------------------------------------------------------------------------|
| <b>Version Control</b> | Git + GitHub, GitLab, Bitbucket                                              |
| <b>CI/CD Server</b>    | Jenkins ("altamente flessibile"), GitHub Actions, GitLab CI/CD, Azure DevOps |
| <b>Build</b>           | dotnet build, Maven, Gradle + <b>Docker CLI</b>                              |
| <b>Registry</b>        | <b>Docker Hub</b> , Amazon ECR, Azure ACR, Google GCR                        |

| Fase       | Strumenti                                                      |
|------------|----------------------------------------------------------------|
| Deploy     | Helm charts, script Ansible                                    |
| Monitoring | Prometheus, Grafana, Datadog, Azure Monitor, Amazon CloudWatch |

*"Non esiste un'unica toolchain corretta. La chiave è automatizzare il processo."*

## Sezione 8 — "Deployment Strategies: Rolling, Blue-Green, Canary"

### ★ 1. Rolling Deployment

Sostituire **GRADUALMENTE** tutte le istanze con nuove istanze, una per una o in piccoli lotti.

v1 v1 v1 v1 v1 → v2 v1 v1 v1 v1 → v2 v2 v1 v1 v1 → ... → v2 v2 v2 v2 v2

*"Si avvia una nuova istanza v2. Una volta che è SANA e pronta ad accettare traffico, l'orchestratore TERMINA una vecchia istanza v1. Ripete finché tutte non sono sostituite."*

*"Il numero totale di istanze disponibili rimane relativamente stabile, così il servizio continua a gestire le richieste senza downtime."*

#### Vantaggio

✓ **"Semplicità e BASSO COSTO di risorse — non serve il doppio dell'infrastruttura" + è la strategia predefinita di Kubernetes**

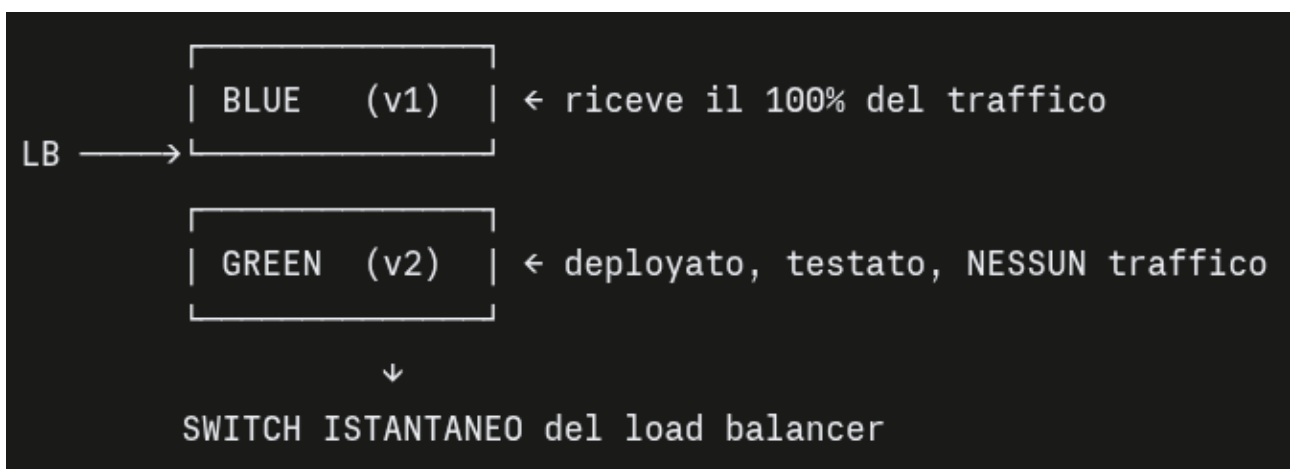
#### Svantaggio

⚠ **"Durante il rollout hai v1 e v2 in esecuzione CONTEMPORANEAMENTE. Problematico se la nuova versione introduce *breaking change* alle API o allo schema del database"**

⚠ **"Il rollback è essenzialmente un altro deployment al contrario — può essere LENTO"**

### ★ 2. Blue-Green Deployment

Due ambienti di produzione **IDENTICI** ma separati: blu e verde.



Il flusso:

1. Blu è live con v1, riceve tutto il traffico
2. **Si distribuisce v2 nell'ambiente verde identico** — *"non riceve ancora traffico utente"*
3. *"Si esegue una serie completa di test sull'ambiente verde, SENZA influenzare gli utenti"*

4. "Quando siamo sicuri, si istruisce il load balancer a passare **ISTANTANEAMENTE** tutto il traffico da blu a verde"
5. "Il vecchio ambiente blu viene mantenuto in standby. Se scopriamo un problema critico, basta invertire il router — rollback immediato"

#### Vantaggio

✓ "Rischio quasi nullo, capacità di cutover e **ROLLBACK Istantanei**"

#### Svantaggio

✗ "Il **COSTO**. È necessario mantenere il **DOPPIO** dell'infrastruttura di produzione — può essere molto costoso"

### ★ 3. Canary Deployment

Rilasciare la nuova versione a un sottoinsieme **MOLTO RISTRETTO** del traffico di produzione.

L'origine del nome:

"Dalla pratica storica **dei minatori di carbone che portavano un canarino nella miniera**. In caso di gas pericolosi, il canarino sarebbe stato colpito per primo, dando ai minatori un allarme tempestivo."

```
1% → [v2] ← il "canarino"
99% → [v1] ← stabile
      ↓ monitoraggio
se OK: 1% → 25% → 50% → 100%
se KO: 100% → v1 istantaneamente
```

"Si configura **il router, l'API Gateway o il SERVICE MESH** per convogliare l'**1% o il 5%** del traffico verso v2."

"Si **monitorano attentamente prestazioni, tassi di errore e METRICHE DI BUSINESS**. Se tutto è a posto, si **aumenta gradualmente la percentuale**. Se si rileva un problema, si **reindirizza immediatamente il 100% del traffico alla v1 stabile**, limitando l'impatto della release difettosa a un piccolo numero di utenti."

#### Vantaggio

✓ ★ "La modalità a **MINOR RISCHIO**. Si può validare una versione con **TRAFFICO DI PRODUZIONE REALE** prima di un rollout completo"

#### Svantaggio

⚠ "Complessità. Richiede **sofisticate capacità di gestione del traffico** — API Gateway o service mesh — e **eccellente monitoraggio e osservabilità in tempo reale**"

Il confronto

|                      | Rolling | Blue-Green          | Canary              |
|----------------------|---------|---------------------|---------------------|
| Costo infrastruttura | ✓ Basso | ✗ 2×                | Medio               |
| Rischio              | Medio   | Basso               | ✓ <b>Minimo</b>     |
| Velocità rollback    | ✗ Lento | ✓ <b>Istantaneo</b> | ✓ <b>Istantaneo</b> |

|                      | Rolling          | Blue-Green        | Canary                          |
|----------------------|------------------|-------------------|---------------------------------|
| Versioni coesistenti | Sì (transitorio) | No (switch netto) | Sì (per design)                 |
| Complessità          | ✓ Bassa          | Media             | ✗ Alta                          |
| Richiede             | Niente           | Doppia infra      | Traffic management + monitoring |

---

## Infrastructure as Code (IaC)

### Definizione

Una pratica di ingegneria del software in cui si gestisce e si fornisce l'infrastruttura — server, reti, database, load balancer — attraverso FILE DI DEFINIZIONE LEGGIBILI DALLA MACCHINA, proprio come si fa con il codice applicativo.

*"Invece di cliccare manualmente sulla console di un cloud provider o eseguire comandi manuali, si definisce l'intera infrastruttura nel codice."*

★ *"L'idea chiave è TRATTARE L'INFRASTRUTTURA COME PARTE DEL SISTEMA SOFTWARE: la sua definizione è memorizzata in Git, le modifiche sono esaminate e approvate tramite pull request, e la distribuzione è automatizzata."*

### I 5 vantaggi

| Vantaggio                   | Dettaglio                                                                                                                                                                                                                                                            |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rapidità di provisioning    | "Un nuovo microservizio ha bisogno di database, cache, code. <b>Si esegue uno script e si fornisce tutta l'infrastruttura necessaria in POCHI MINUTI</b> "                                                                                                           |
| Coerenza tra ambienti ★     | "Garantisce che dev, staging e produzione siano creati dalla <b>STESSA IDENTICA</b> definizione. Riduce drasticamente i bug che derivano da sottili differenze tra ambienti"                                                                                         |
| Infrastruttura immutabile ★ | "Invece di <b>aggiornare o patchare i server in esecuzione</b> — che porta a configuration drift e stati imprevedibili — si fornisce <b>NUOVA infrastruttura con la configurazione aggiornata, si distribuisce, si sposta il traffico, e si TERMINA la vecchia</b> " |
| Versionamento e rollback    | "Avete <b>una storia completa di ogni modifica. Se una modifica introduce un problema, si torna a una versione stabile precedente</b> — proprio come col codice"                                                                                                     |
| Integrazione con CI/CD      | "La pipeline può <b>fornire automaticamente l'infrastruttura, distribuire, testare, e poi SMANTELLARE l'infrastruttura</b> "                                                                                                                                         |

**Definizione: configuration drift** — la divergenza progressiva tra la configurazione reale dei server e quella documentata, causata da modifiche manuali accumulate nel tempo. È la ragione per cui "l'ambiente di staging non si comporta come produzione".

### Gli strumenti

| Categoria                   | Strumenti                                                                                                                                                                                                    |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Provisioning cloud-agnostic | ★ <b>Terraform</b> ("chiaramente il leader del settore", dichiarativo, multi-cloud), <b>Pulumi</b> ("consente di definire l'infrastruttura usando <b>linguaggi familiari come C#, Python o TypeScript</b> ") |
| Configuration               | <b>Ansible, Chef, Puppet</b>                                                                                                                                                                                 |

| Categoria    | Strumenti                                                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------------------------|
| management   |                                                                                                                       |
| Cloud-native | AWS: <b>CloudFormation</b> , SAM, CDK. Azure: <b>ARM template</b> , Bicep.<br>Google: <b>Cloud Deployment Manager</b> |

### ★ Il workflow di Terraform

1. DEFINE → file .tf in HCL (HashiCorp Configuration Language)
2. terraform init → scarica i plugin dei provider, imposta il backend di stato
3. terraform plan ★ → MOSTRA quali modifiche verranno apportate
4. terraform apply → crea/modifica le risorse per corrispondere allo stato desiderato
- ...
5. terraform destroy → rimuove tutte le risorse (utile in dev/test per non sprecare costi)

### La gestione dello stato:

*"Terraform tiene traccia dello stato della vostra infrastruttura in un file di stato, che usa per pianificare i passi necessari. È importante gestirlo con attenzione, soprattutto in un ambiente di squadra."*

### Due best practice finali:

1. *"Memorizzate SEMPRE i file di configurazione in un sistema di controllo delle versioni."*
2. *"Tenete i dati sensibili FUORI dai file di configurazione, usando variabili di input o sistemi di gestione dei segreti."*

## What is GitOps?

### Definizione

**Un approccio moderno alla continuous delivery che usa IL REPOSITORY GIT COME UNICA FONTE DI VERITÀ per descrivere lo stato desiderato dell'INTERO SISTEMA.**

*"Include sia la configurazione dell'applicazione — quale immagine eseguire — sia le definizioni dell'infrastruttura — manifest Kubernetes, file Terraform."*

*"Applica le stesse best practice che usiamo per le applicazioni — versionamento, pull request, revisione tra pari, automazione — direttamente alla gestione del nostro AMBIENTE DI PRODUZIONE."*

### ★ La differenza chiave: PUSH vs PULL

|           | CI/CD tradizionale                                | GitOps                                                                                          |
|-----------|---------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Direzione | <b>PUSH</b> — "la pipeline SPINGE le modifiche al | <b>PULL</b> — "un agente IN ESECUZIONE DENTRO il cluster ESTRAE lo stato desiderato da Git e lo |

|                    | CI/CD tradizionale                                        | GitOps                                                                |
|--------------------|-----------------------------------------------------------|-----------------------------------------------------------------------|
|                    | <i>cluster"</i>                                           | <i>applica"</i>                                                       |
| <b>Credenziali</b> | ⚠️ La pipeline ha le credenziali per scrivere sul cluster | ✅ <b>"Più sicuro":</b> il cluster non espone credenziali all'esterno" |
|                    | CI/CD: [Pipeline] —push credenziali—> [Kubernetes]        |                                                                       |
|                    |                                                           | GitOps: [Git] ←pull← [Agente dentro Kubernetes]                       |

## I 4 principi di GitOps

### 1. Configurazione dichiarativa

*"L'intero stato desiderato è descritto in modo dichiarativo: **manifest YAML o JSON** per Deployment, Service, ConfigMap."*

### 2. Git come unica fonte di verità ★

*"Il repository Git è l'**UNICO** luogo in cui questo stato viene definito e modificato. **NON** è possibile apportare modifiche manuali direttamente al cluster di produzione usando **kubectl**. Se si vuole fare una modifica, bisogna prima farla in Git."*

*"In questo modo si ottiene **una cronologia IMMUTABILE e VERIFICABILE** di ogni modifica."*

### 3. Deployment automatizzato pull-based

*"Un agente in esecuzione **DENTRO** il cluster monitora il repository Git. Quando rileva che la configurazione è cambiata, estrae le modifiche e le applica."*

### 4. Riconciliazione continua ★

*"L'agente **NON SOLO** applica le modifiche: **CONFRONTA CONTINUAMENTE** lo stato live del cluster con lo stato desiderato in Git. Se ci sono differenze, **CORREGGE AUTOMATICAMENTE** lo stato live, garantendo autoguarigione e coerenza."*

Questo è il punto più potente: se qualcuno modifica manualmente il cluster, **GitOps annulla la modifica**. Il drift diventa impossibile.

## Il workflow in 4 fasi

1. DEFINE → il team definisce lo stato desiderato in file dichiarativi su Git
2. PULL REQUEST → chi vuole una modifica apre una PR
  - il team la REVISIONA, dà feedback, approva o rifiuta
3. SYNC → al merge, l'OPERATORE GitOps rileva la modifica e la applica al cluster
  - confronta continuamente e sincronizza
4. MONITOR → logging, monitoring, alerting se lo stato devia

## Il caso d'uso Black Friday

*"Il team decide di scalare i servizi per gestire l'aumento di traffico, concentrandosi sulla parte più trafficata: il microservizio dei prodotti."*

### Il flusso:

1. **Definizione:** ogni microservizio è definito in file dichiarativi in Git, insieme all'infrastruttura (load balancer, policy di rete)
2. **Pull request:** uno sviluppatore **modifica i file di configurazione aumentando il numero di repliche** e apre una PR

*"Il team **esamina le modifiche:** il processo di revisione **assicura che tutti siano consapevoli di ciò che viene modificato** e possano **verificare che siano seguiti best practice e standard**"*

3. **Sincronizzazione automatica:** *"dopo l'approvazione e il merge, un operatore GitOps come Argo CD o Flux rileva la modifica e la applica automaticamente, aumentando il numero di repliche"*
4. **Monitoraggio:** *"se si verifica un problema, il team viene avvisato. Un hotfix si fa modificando direttamente la configurazione in Git, e l'operatore lo applica"*

### Rollback e audit trail ★:

*"Supponiamo che questa modifica causi un problema. Poiché **TUTTE le modifiche sono versionate in Git**, il team può tornare rapidamente alla configurazione precedente ripristinando le modifiche nel repository. E questa azione verrebbe automaticamente riflessa nel cluster dall'operatore GitOps."*

**Strumenti GitOps:** Argo CD, Flux, Jenkins X, Tekton. Gestiti: AWS Proton, Azure Arc, Google Config Sync.

---

## Design: E-Shop Microservices with DevOps, CI/CD, IaC and GitOps

### Toolkit aggiornato

**Nuovi requisiti non funzionali:** container e orchestratori per packaging e deployment; **cultura DevOps** abilitata da **pipeline CI/CD**; **Infrastructure as Code**; **GitOps** con Git come unica fonte di verità.

**Nuovi pattern:** Helm charts, pattern sidecar, service mesh, strategie di deployment (blue-green, rolling, canary).

### ★ L'architettura finale — i 7 livelli

1. CONTAINERIZZAZIONE → ogni microservizio = container Docker
2. ORCHESTRAZIONE → deployati su cluster Kubernetes (pod, auto-scaling, self-healing)
3. PACKAGING → Helm charts gestiscono la complessità dei file YAML
4. SERVICE MESH → Istio/Linkerd + sidecar per traffico, sicurezza, osservabilità

5. AUTOMAZIONE → pipeline CI/CD per ogni microservizio
6. DEPLOYMENT STRATEGY → canary o blue-green per zero downtime
7. IaC + GitOps → Terraform definisce l'infra; Argo CD sincronizza da Git

*"Quando uno sviluppatore spinge il codice per Catalog nel repository Git, una pipeline CI/CD dedicata costruisce automaticamente l'immagine, la testa, la spinge nel registry ed esegue il deployment nel cluster."*

*"Invece della pipeline che spinge le modifiche direttamente su Kubernetes, usiamo il workflow GitOps: se inviamo una PR per aggiornare la versione di un'immagine, una volta unita, un agente GitOps come Argo CD in esecuzione DENTRO il cluster rileva la modifica, estrae la nuova configurazione e la applica automaticamente."*

### ★ Lo stack tecnologico concreto (esempio AWS)

| Componente    | Scelta                                                                                                                                              |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Container     | Docker (o Podman)                                                                                                                                   |
| Orchestratore | Amazon EKS (Kubernetes gestito)                                                                                                                     |
| Registry      | Amazon ECR                                                                                                                                          |
| Packaging     | Helm charts                                                                                                                                         |
| CI/CD         | GitHub Actions — <i>"si attiva su modifiche al codice, costruisce l'immagine Docker, la spinge su ECR, e <b>AGGIORNA I MANIFEST KUBERNETES</b>"</i> |
| IaC           | Terraform — <i>"definisce cluster EKS, rete, repository ECR, ruoli e policy IAM"</i>                                                                |
| GitOps        | Argo CD — <i>"deployato nel cluster EKS, osserva i repository GitHub per modifiche ai manifest e li distribuisce automaticamente"</i>               |
| Service Mesh  | Istio o Linkerd                                                                                                                                     |
| Monitoring    | Prometheus + Grafana o AWS CloudWatch                                                                                                               |

### ★ Il workflow completo end-to-end

1. PROVISIONING → terraform init / plan / apply → crea EKS, ECR, rete
2. SETUP GITOPS → deploy Argo CD nel cluster, configurato per osservare GitHub
3. SVILUPPO → il team scrive il codice e fa push su GitHub
4. CI → GitHub Actions si attiva:
  - └─ costruisce l'immagine Docker
  - └─ la invia a ECR
  - └─ **AGGIORNA** i manifest Kubernetes con il nuovo tag
5. CD (GitOps) → Argo CD rileva la modifica ai manifest
  - └─ estrae e applica al cluster EKS
6. MONITORING → Prometheus/Grafana osservano prestazioni e salute

**Nota la separazione elegante:** la pipeline CI **non tocca mai il cluster**. Si limita a **scrivere su Git**. Argo CD, dentro il cluster, fa il resto. Questo è il cuore di GitOps.

---

## Code: Deploy to Azure Container Apps with azd

Qui il corso mostra un percorso **più semplice** di Kubernetes puro: **Azure Container Apps** con **.NET Aspire** e l'**Azure Developer CLI (azd)**.

*"Questo approccio **riduce drasticamente la complessità del provisioning di singole risorse Azure** — Container Apps, Container Registry, Log Analytics."*

### I prerequisiti

1. **Abbonamento Azure** (va bene la prova gratuita)
2. **Azure CLI** e **Azure Developer CLI (azd)** installati
3. Il progetto **.NET Aspire** da distribuire

### I comandi

```
azd version      # verifica l'installazione
azd auth login    # login su Azure
cd <cartella-soluzione-Aspire>
azd init          # ★ SCANSIONA la directory e RILEVA il progetto Aspire
                  # genera un file YAML che descrive come
                  # costruire e distribuire i microservizi
azd up           # ★ provisioning + deployment
                  # chiede: subscription + region
                  # → richiede 5-15 minuti
```

*"azd init esegue la scansione della nostra applicazione e comprende il progetto Aspire. Ha rilevato il nostro progetto Aspire, e genera il file YAML creato da Azure che descrive come costruire e distribuire le nostre applicazioni."*

*"azd up: ogni microservizio viene avviato come Container App, e anche i backing services — Redis, RabbitMQ — vengono configurati nel resource group."*

### Il risultato

Nel portale Azure, dentro il nuovo **resource group**, compaiono:

- **Container Registry** (per le immagini)
- **Log Analytics** (per i log)
- **Container Apps Environment**
- **Le Container App** dei microservizi: cache, catalog, keycloak, postgresql, rabbitmq, basket, e la **webapp Blazor**

## ★ Il dettaglio dell'esposizione: **internal**

*"Le parole chiave **"internal"** sono incluse SOLO nei backing services e nei microservizi interni. La webapp NON include la parola chiave internal, perché esponiamo questo servizio all'esterno, scrivendo endpoint HTTP esterni nel nostro progetto."*

Questo riflette esattamente il pattern **API Gateway** della Lezione 8: **solo un componente è pubblico; tutto il resto è raggiungibile solo dall'interno della rete.**

## ★ Le variabili d'ambiente iniettate

Apriamo la Container App di **Basket** → Containers → Environment variables:

*"Si vedono **gli endpoint per il SERVICE DISCOVERY**: l'integrazione con Catalog e Keycloak dal microservizio Basket. I valori sono **GENERATI dalle Azure Container Apps**, compresi i collegamenti interni."*

*"Questi sono i valori di service discovery iniettati da .NET Aspire, e si riflettono sulle Azure Container Apps."*

*"Scorrendo in basso si vedono **due connection string** per connettersi ai backing services — **cache distribuita e RabbitMQ** — e queste sono memorizzate come **SECRET** nelle Azure Container Apps."*

★ *"Questa è la potenza di .NET Aspire: tutte queste configurazioni sono gestite dal processo di distribuzione."*

Nota quanto è importante: **il modello di service discovery che abbiamo usato in locale dalla Lezione 5 si traduce automaticamente nella configurazione cloud.** Nessuna riscrittura.

## Il test finale

1. Si apre la **dashboard Aspire in cloud** (elenca tutte le risorse deployate)
2. Si apre la **webapp Blazor** — *"solo la webapp può essere esportata verso sistemi esterni"*
3. Si clicca **Products** → i prodotti vengono elencati dal microservizio Catalog ✓
4. **Nelle tracce della dashboard**: la prima chiamata colpisce `/products` → integra con Catalog → **scrive i dati in Redis per l'output cache**
5. **Alla seconda chiamata**: *"la risposta proviene solo dalla cache Redis, che ha eseguito con successo l'output caching"* ✓

Nota il bel riepilogo implicito: la Lezione 3 (output caching) funziona identica in produzione cloud.

---

## Problem: Fault Tolerance — Microservices Must Remain Operational

### Cosa abbiamo ottenuto

*"I container vengono eseguiti **in modo coerente ovunque**; i microservizi sono **portabili e isolati**. Kubernetes offre **eccellente scalabilità e alta disponibilità a livello di***

**INFRASTRUTTURA** — *può riavviare automaticamente un container se si blocca. Il processo di distribuzione è automatizzato e affidabile con CI/CD e GitOps.*"

### ★ Il limite di Kubernetes

*"Mentre Kubernetes può riavviare un container in avaria, NON PUÒ risolvere magicamente un bug nel codice dell'applicazione, né PREVENIRE UN GUASTO A CASCATA causato da una connessione di rete lenta tra i microservizi."*

Questa è l'osservazione chiave: **Kubernetes gestisce i guasti dei processi, non i guasti delle interazioni.**

### ★ Lo scenario: il fallimento a cascata

*"Cosa succede durante i saldi del Black Friday, se il nostro servizio Discount diventa LENTO o non risponde?"*

```
[Discount] diventa lento
  ↓
Ogni chiamata da [Basket] diventa lenta o va in timeout
  ↓
I THREAD di Basket restano BLOCCATI, in attesa di una risposta che potrebbe non
arrivare mai
  ↓
TUTTI i thread disponibili in Basket vengono consumati
  ↓
Basket non può rispondere a NESSUNA richiesta — ANCHE QUELLE NON CORRELATE
  ↓
Il guasto si propaga a Ordering, alla WebApp...
  ↓
🔥 CASCADING FAILURE
```

★ *"Un guasto in un SEMPLICE microservizio può innescare un GUASTO A CASCATA, che porta all'arresto di parti importanti dell'applicazione e alla perdita di fatturato e di fiducia dei clienti."*

**Il dettaglio più insidioso:** il servizio lento **non è nemmeno down**. Se fosse down, la chiamata fallirebbe subito. **È la LENTEZZA che uccide**, perché consuma risorse del chiamante senza restituire nulla.

### I 3 punti di guasto della rete

| Tipo                         | Descrizione                                                                                           |
|------------------------------|-------------------------------------------------------------------------------------------------------|
| Errori transitori di rete    | "Cadute temporanee di connessione o picchi di latenza"                                                |
| Indisponibilità del servizio | "Un servizio può essere <b>temporaneamente offline</b> per un riavvio, o bloccarsi a causa di un bug" |
| Sovraccarico del servizio    | "Un servizio potrebbe essere <b>sommerso di richieste</b> e diventare molto lento nel rispondere"     |

## L'esigenza

*"La nostra architettura **DEVE ESSERE RESILIENTE**: deve essere in grado di resistere a questi guasti e riprendersi **CON GRAZIA**, garantendo che **il sistema complessivo rimanga operativo e disponibile**."*

## La soluzione

*"Come soluzione, impareremo:"*

- **pattern di resilienza e fault tolerance** per i microservizi
  - **osservabilità dei microservizi: logging distribuito, tracing distribuito, monitoraggio della salute**
-